

Market by price (MBP-1)

MBP-1 (market by price) provides every order book event that updates the top price level, also known as the best bid and offer (BBO). This includes every trade and changes to book depth, alongside total size and order count at the BBO.

This is often called "L1 data".

Databento groups this under [top of book \(L1\)](#).

Fields (`mbp-1`)

Field	Type	Description
<code>ts_recv</code>	<code>uint64_t</code>	The capture-server-received timestamp expressed as the number of nanoseconds since the UNIX epoch. See ts_recv .
<code>ts_event</code>	<code>uint64_t</code>	The matching-engine-received timestamp expressed as the number of nanoseconds since the UNIX epoch. See ts_event .
<code>rtype</code>	<code>uint8_t</code>	A sentinel value indicating the record type. Always 1 in the MBP-1 schema. See Rtype .
<code>publisher_id</code>	<code>uint16_t</code>	The publisher ID assigned by Databento, which denotes the dataset and venue. See Publishers .
<code>instrument_id</code>	<code>uint32_t</code>	The numeric instrument ID. See Instrument identifiers .
<code>action</code>	<code>char</code>	The event action. Can be Add , Cancel , Modify , clear book, or Trade . See Action .
<code>side</code>	<code>char</code>	The side that initiates the event. Can be Ask for a sell order (or sell aggressor in a trade), Bid for a buy order (or buy aggressor in a trade), or None where no side is specified. See Side .
<code>depth</code>	<code>uint8_t</code>	The book level where the update event occurred.
<code>price</code>	<code>int64_t</code>	The order price where every 1 unit corresponds to 1e-9, i.e. 1/1,000,000,000 or 0.000000001. See Prices .
<code>size</code>	<code>uint32_t</code>	The order quantity.
<code>flags</code>	<code>uint8_t</code>	A bit field indicating event end, message characteristics, and data quality. See Flags .
<code>ts_in_delta</code>	<code>int32_t</code>	The matching-engine-sending timestamp expressed as the number of nanoseconds before <code>ts_recv</code> . See ts_in_delta .
<code>sequence</code>	<code>uint32_t</code>	The message sequence number assigned at the venue.
<code>bid_px_00</code>	<code>int64_t</code>	The bid price at the top level where every 1 unit corresponds to 1e-9, i.e. 1/1,000,000,000 or 0.000000001. See Prices .
<code>ask_px_00</code>	<code>int64_t</code>	The ask price at the top level where every 1 unit corresponds to 1e-9, i.e. 1/1,000,000,000 or 0.000000001. See Prices .
<code>bid_sz_00</code>	<code>uint32_t</code>	The bid size at the top level.
<code>ask_sz_00</code>	<code>uint32_t</code>	The ask size at the top level.
<code>bid_ct_00</code>	<code>uint32_t</code>	The bid order count at the top level.
<code>ask_ct_00</code>	<code>uint32_t</code>	The ask order count at the top level.

See also

Some venues or data feeds may publish two-sided BBO changes in a single update. In cases like this where the `side` is indeterminate, we'll also use the side code `N`. Exceptions and edge cases like this can be found in our [Venues and datasets](#) section.

Implementation differences between clients and encodings

Bid and ask depth messages (fields starting with `bid_` and `ask_`) are structured differently in the **C++ and Rust clients**, the **Python record interface**, and **JSON data**. Instead of using the `_N` suffix, they're stored in an array of structures named `levels`, with the top-of-book at index `0`.

For example, in C++, `levels[5].bid_px` corresponds to `bid_px_05` in the Python DataFrame API and CSV format.

See also

MBP-1 has many similarities to the TBBO and BBO schemas. The main distinction is that MBP-1 is in book update space, while TBBO is in trade space, and BBO is in time space. Learn about the differences between each in our [MBP-1 vs. TBBO vs. BBO schemas guide](#).